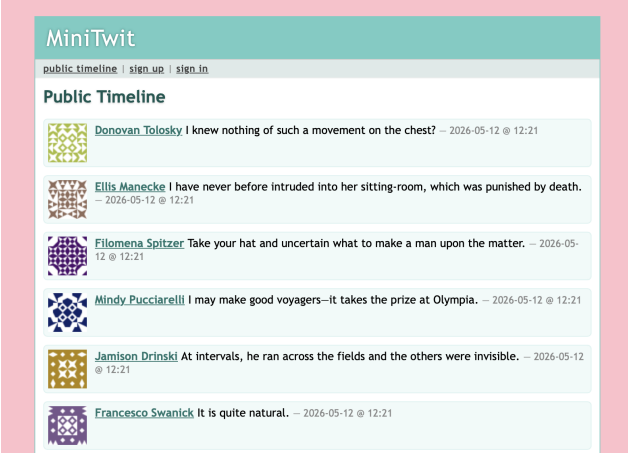
DevOps, Software Evolution and Software Maintenance, MSc



Group K - Pat Myaz

Carmen Alberte Nielsen | alcn@itu.dk
Casper Storm Frøding | csfr@itu.dk
Mads Christian Nørklit Jensen | macj@itu.dk
Mathilde Julie Gonzalez-Knudsen | mgon@itu.dk
Max Brix Koch | mbko@itu.dk

18-05-2026

Contents

1 System's Perspective **2**
1.1 Design and Architecture . . . 2
1.2 Dependencies . . . 2
1.3 Current State . . . 2

2 Process' Perspective **2**
2.1 CI/CD . . . 2
2.2 Monitor . . . 2
2.2.1 Dashboards . . . 3
2.3 Logging . . . 4
2.4 Security . . . 4
2.5 Availability . . . 4

3 Reflection Perspective **4**
3.1 Evolution and Refactoring . . . 4
3.2 Operation . . . 4
3.3 Maintenance . . . 4

4 Use of AI **4**

1 System's Perspective

1.1 Design and Architecture

1.2 Dependencies

1.3 Current State

2 Process' Perspective

2.1 CI/CD

2.2 Monitor

Under here is a sequence diagram of the process of collecting, forwarding, and visualizing application metrics in the monitoring system. A sequence diagram of how monitoring works for Minitwit

To ensure we do not leak information on our metrics of Minitwit, we have set a username and password on Grafana, that is automatically set on creation. In addition to this, only the Monitor's droplet IP adress is whitelisted to scrape https://monitor.bigtwit.app/, done via Traefik, so no other attacker would be able to access this information.

On this metrics endpoint we use `app.UseHttpMetrics();` , that out of the box, write things to the endpoint like total amount of HTTP requests recieved, the duration of these requests, and more HTTP specific metrics. In addition to this, we added 2 "gauges" to the metrics, that showed the amount of tweets and users the app has. This data is cached, so if another metrics request is recieved in less than 30 seconds, we re-use the old data for tweets, and followers. This is done to not overwork the database, with requests to count the whole amount of tweets and users. The source code for caching, in the metrics As can be seen from this code and the Sequence diagram, we have a hybrid of push/pull monitoring, as Minitwit pushes some internal data to a metrics endpoint, and Promtheus pulls from this endpoint.

As can be seen in the diagram, Traefik load balances between each container, which creates this fluctuating graph, that can be seen in the dashboard under here, due to Prometheus not knowing exactly what container it has scraped, and can therefore not make an average.

To mitigate this error, we should have moved our Prometheus container to the Manager droplet, and then from here it would be able to scrape all the individual replicas, and find averages over these, via the Docker network, and communicate this to Grafana on the monitoring droplet. However, this would still be a problem due to the workers being on different droplets than the manager, therefore we would need a package that could discover the exact location of each replica, for example this package: HERE.

In addition to this we also wanted to monitor all of our services, i.e. not "just" the webservice, but also the database, to make it easier to determine where a error/bottleneck is located, however we did not get to implement this.

Due to us developers not wanting to constantly monitor our system, we have set up a *Uptime Check*, that monitors our droplets, and gives everyone in the team a notification if: * https://bigtwit.app is down * there is latency over 1000ms when accessing https://bigtwit.app * The SSL certificate of bigtwit.app is about to expire.(However Traefik should automatically renew this certificate, before they expire.)

In addition to this we have added a ressource alert, that monitors our test database and production database, to see when it crosses over 70% CPU in over 5 minutes, which could indicate an attack, or a rapid increase in requests that it maybe cannot handle.

2.2.1 Dashboards

To see the dashboard, go to 46.101.69.11:3000 , where the dashboard + logs can be viewed. The credentials is specified in the post made by Helge.

We made 2 different dashboards, one made with a developer focus(Infrastructure Health), and one made for a Business stakeholder focus(Product Performance).

2.2.1.1 Minitwit Infrastructure Health, dashboard

The first part of the Minitwit Infrastructure Health dashboard The second part of the Minitwit Infrastructure Health dashboard The third part of the Minitwit Infrastructure Health dashboard

This dashboard give the developers information they need like: - Immediate Root Cause Analysis: If the Status is UP but the Error Rate is spiking, developers know the app is running but the code or database is failing. - Capacity Planning: Requests per second and Total Requests help developers understand if the current infrastructure (CPU/RAM) can handle the load or if they need to scale. - Performance Regressions: By monitoring Latency(P95) and HTTP Success rate, developers can see if a recent code deployment made the app slower, even if it didn't *break* it.

We have, in the source code, created a */healthz* endpoint, that shows when minitwit is up and running, however this is only used for spinning up the system, and our dashboard's health check is a health check of Prometheus was able to scrape the target. If we had more time, we would have used the */healthz* endpoint instead, which would give a more relvant answer, i.e. seperated from Prometheus.

2.2.1.2 Minitwit Product Performance, dashboard The first part of the Minitwit Infrastructure Health dashboard The second part of the Minitwit Infrastructure Health dashboard The third part of the Minitwit Infrastructure Health dashboard

- KPI Tracking: "Total Tweets" and "amount of users" are direct Key Performance Indicators. In a real application, if registrations drop to zero, the Marketing or Product team needs to know immediately.
- Engagement Trends: The 24-hour tweet increase shows the pulse of the community. A sudden drop might indicate a trend issue or a subtle bug that doesn't trigger a error but prevents posting. Also, amount of users registerered in the last hour, indicates if a campaign or other trend has worked or not, and if we therefore can see an increase or a drop.
- User Satisfaction: Slow apps lead to lost users. Including it here keeps the business aligned with the need for technical optimization.

2.3 Logging

2.4 Security

2.5 Availability

3 Reflection Perspective

3.1 Evolution and Refactoring

Remember to write about how changing from one docker container, to using Docker swarm destroyed our old way of monitoring. (OBS! we have written something about monitoring, so just a quick comment on how this destroyed it.)

3.2 Operation

3.3 Maintenance

4 Use of AI